



دانشگاه صنعتی شیراز

انجمن علمی کامپیوتر
دانشگاه صنعتی شیراز

سایبر سوکت

نشریه علمی دانشجویی انجمن علمی کامپیوتر دانشگاه صنعتی شیراز

شماره سوم، تابستان 1405

ویژه نامه

چطور یک

چت بات

هوشمند با اطلاعات شخصی

بسازیم؟



Smart
AI
Better
Together



</>



ویژه نامه سایبرسوتک

● سال دوم | شماره سوم | تابستان ۱۴۰۵ | شماره مجوز: ف/ن/ف/۱۱

● صاحب امتیاز: انجمن علمی مهندسی کامپیوتر دانشگاه صنعتی شیراز

ما را در فضای مجازی دنبال کنید



Aparat



Linkedin



Gmail



Telegram



Insta-



Github

در نسخه الکترونیکی نشریه هرگاه این علامت را دیدید، با کلیک بر روی آن میتوانید به لینک مربوطه دسترسی پیدا کنید

ویژه نامه سایبرسوتک

دبیر انجمن علمی



محمد اشتیاق
مهندسی کامپیوتر

مدیر مسئول نشریه



محمد رضایی
مهندسی برق

سرمدیر نشریه



امیرحسین خدیوی
مهندسی کامپیوتر

نویسندگان
ویژه نامه



میبینا خردمند
مهندسی کامپیوتر



یلدا موسوی
مهندسی کامپیوتر



اشکان رفیعی نژاد
مهندسی کامپیوتر



هدیه حبیبی
مهندسی کامپیوتر

ویراستاری



فاطمه رضوی
مهندسی کامپیوتر



حدیث خادم حمزه
مهندسی کامپیوتر

گرافیکست و
صفحه آرایی



یاسمن بنایی
مهندسی کامپیوتر



نیلوفر سلطان
مهندسی کامپیوتر



فاطمه حسینی
مهندسی کامپیوتر



فاطمیما خندستانی
مهندسی کامپیوتر

فضای مجازی



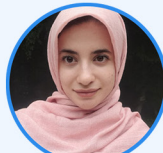
فاطمه حسن زاده
مهندسی کامپیوتر



یاسمن چغانی
مهندسی کامپیوتر



فاطمه یزدانی
مهندسی کامپیوتر



سمیه جهانین
مهندسی کامپیوتر

سخن سردبیر

با سلام و احترام

با توجه به رشد روزافزون هوش مصنوعی‌های مختلف از نظر تعداد، قدرت و تخصص، تصمیم گرفتیم که بصورت اختصاصی در یک ویژه‌نامه جداگانه دستیارهای هوشمندی را که می‌توانیم امروزه از آنها برای عملکرد بهتر و سریع‌تر در کارهای روزمره خود استفاده کنیم، معرفی نماییم و نحوه ساختن یکی از این موارد را آموزش دهیم.

نشریه سایبرسوتک با تشکیل تیمی جدید با عنوان تیم ویژه‌نامه قصد دارد زین پس برای مطالب اختصاصی و ترندهای روز یا مطالب کاربردی بصورت موازی با فصلنامه مجله‌ای با یکی از عناوین ذکر شده را ارائه دهد و برای درک راحت‌تر مخاطب بعد از بیان تئوری مطالب، با مثال و پروژه‌های عملی کاربرد مطالب گفته شده در مجله را در دنیای علمی و صنعتی نمایش دهد. در پایان، از تلاش و همراهی تمامی همکارانی که در تهیه و انتشار این شماره مشارکت داشتند و همچنین شما مخاطبان گرامی، صمیمانه سپاسگزارم.

امیرحسین خدیوی خانقاه

سردبیر نشریه سایبر سوتک





چگونه یک چت بات هوشمند با اطلاعات شخصی بسازیم؟

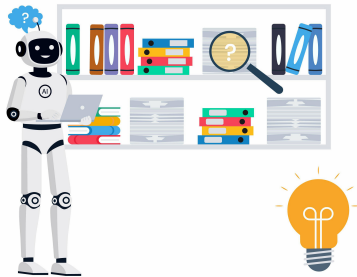
مقدمه

اگر یک مدل زبانی می‌تواند درباره‌ی میلیون‌ها موضوع صحبت کند، پس چرا نمی‌تواند درباره‌ی جزوه‌ای که همین حالا جلوی شماست به سؤال‌ها پتان جواب بدهد؟ فرض کنید چندصد صفحه جزوه دانشگاهی، مجموعه‌ای از اسناد یک شرکت یا سازمان، یا حتی یک آرشیو بزرگ از اطلاعات در اختیار دارید و می‌خواهید از یک مدل زبانی درباره‌ی محتوای آن‌ها سؤال بپرسید. سؤال ساده است؛ اما پاسخ به آن چندان ساده نیست. مدل از کجا باید بداند اطلاعات موردنیاز شما دقیقاً در کدام صفحه، کدام پاراگراف یا کدام

بخش از این حجم عظیم محتوا قرار گرفته است؟ اینجاست که یکی از مشکلات مهم استفاده از مدل‌های زبانی بزرگ (LLM) خودش را نشان می‌دهد: ممکن است مدل پاسخی بدهد که از نظر زبانی کاملاً درست، روان و حتی قانع‌کننده به نظر برسد، اما لزوماً پاسخ سؤال ما نباشد. این اتفاق می‌تواند دلایل مختلفی داشته باشد؛ مثلاً ممکن است اطلاعات موردنیاز برای پاسخ‌گویی اصلاً در داده‌های آموزشی مدل وجود نداشته باشد. LLM ها روی حجم بسیار زیادی از داده‌ها

آموزش دیده‌اند و می‌توانند متن تولید کنند، به سؤال‌ها جواب دهند، متن را خلاصه کنند یا حتی کد بنویسند. اما این توانایی‌ها به این معنا نیست که همیشه به اطلاعات موردنیاز ما دسترسی دارند. مدل زبانی به‌طور معمول به جزوه شخصی شما یا اسناد داخلی یک شرکت دسترسی ندارد. از طرف دیگر، حتی اگر تمام محتوای اسناد را یک‌جا در اختیار مدل قرار دهیم، باز هم مسئله کاملاً حل نمی‌شود. وقتی حجم اطلاعات زیاد باشد، پیدا کردن و استفاده از بخش‌های مرتبط برای مدل دشوارتر می‌شود. پس شاید راهکار بهتر این باشد که به‌جای ارائه‌ی تمام اطلاعات، ابتدا بخش‌هایی را که بیشترین ارتباط را با سؤال دارند، از میان منابع پیدا کنیم و بعد همان بخش‌ها را در اختیار مدل قرار دهیم. ایده اصلی (Retrieval-Augment-Generation) RAG در ابتدا اطلاعات مرتبط با سؤال کاربر (Retrieval) می‌شوند و بعد این اطلاعات در اختیار مدل زبانی قرار می‌گیرند تا بر اساس آن‌ها پاسخ را تولید (-Generation) کند. به این ترتیب، مدل به‌جای این‌که فقط از اطلاعاتی که در زمان آموزش یاد گرفته استفاده کند، می‌تواند هنگام پاسخ‌گویی از

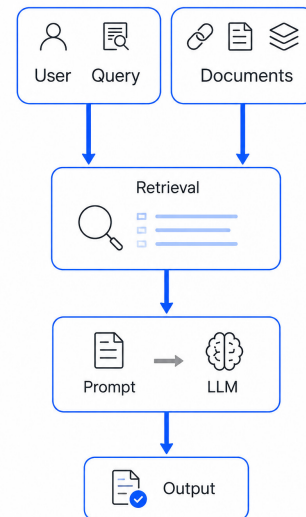
اطلاعات مرتبط و خارجی هم کمک بگیرد.



RAG چطور پاسخ را پیدا می‌کند؟

حالا که با ایده‌ی اصلی RAG آشنا شدیم، وقت آن است یک قدم جلوتر برویم و ببینیم پشت صحنه چه اتفاقی می‌افتد. RAG چطور از میان انبوهی از اطلاعات، بخش مرتبط با سؤال ما را پیدا می‌کند و آن را به یک پاسخ قابل‌استفاده تبدیل می‌کند؟ اگر بخواهیم کل این فرایند را خیلی ساده در نظر بگیریم، می‌توانیم آن را در سه مرحله خلاصه کنیم: آماده‌سازی اطلاعات، بازیابی اطلاعات مرتبط و تولید پاسخ. در ابتدا، منابعی که قرار است مورد جست‌وجو قرار بگیرند آماده می‌شوند. سپس اطلاعات مرتبط با سؤال کاربر از میان این منابع پیدا می‌شوند و در نهایت، مدل زبانی با استفاده از همین بخش‌های مرتبط، پاسخ نهایی را تولید می‌کند.





وقتی اطلاعات آماده‌ی جست‌وجو نیستند، چه می‌شود؟

قبل از آن‌که سیستم بتواند اطلاعات موردنیاز را از میان اسناد پیدا کند، منابع باید به شکلی آماده شوند که امکان جست‌وجو در آن‌ها وجود داشته باشد.

برای مثال، یک فایل PDF ممکن است ده‌ها یا حتی صدها صفحه داشته باشد. طبیعتاً نمی‌توان انتظار داشت که هنگام پاسخ‌گویی، همیشه کل این فایل در اختیار مدل قرار بگیرد. گذشته از حجم بالای اطلاعات، چنین کاری می‌تواند فرایند پیدا کردن بخش‌های مرتبط را نیز دشوارتر کند.

مدل‌های زبانی برای هر درخواست، ظرفیت مشخصی برای دریافت اطلاعات دارند که با عنوان Context Window شناخته می‌شود. بنابراین، وقتی با اسناد حجیم سروکار داریم، نمی‌توان تمام محتوای آن‌ها را به صورت هم‌زمان در اختیار مدل قرار داد.

متن را چطور به قطعه‌های قابل جست‌وجو تبدیل می‌کنیم؟

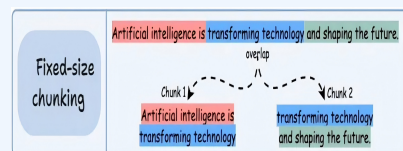
یکی از مهم‌ترین کارها در این مرحله، تقسیم اسناد به بخش‌های کوچک‌تر، به نام Chunk است.

شاید در نگاه اول، Chunking فقط به معنای تکه‌تکه کردن متن به نظر برسد؛ اما ماجرا به همین سادگی نیست. این‌که متن دقیقاً از کجا و با چه روشی تقسیم شود، می‌تواند مستقیماً بر کیفیت بازیابی اطلاعات تأثیر بگذارد.

یکی از ساده‌ترین روش‌ها برای این کار، Fixed-Size Chunking است. ایده بسیار ساده است:

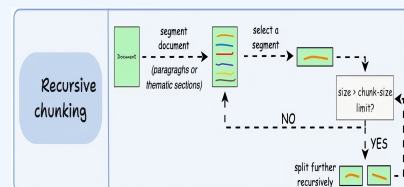
متن را به قسمت‌هایی با اندازه‌ی مشخص تقسیم می‌کنیم و همین روند را تا پایان ادامه می‌دهیم. برای مثال، می‌توانیم تعیین کنیم که هر Chunk شامل ۵۰۰ کاراکتر باشد. اما این روش یک مشکل مهم دارد. ممکن است

اطلاعاتی که در مرز دو بخش قرار گرفته‌اند، از یکدیگر جدا شوند؛ برای مثال، ممکن است یک جمله درست جایی قطع شود که ادامه‌ی آن در Chunk بعدی قرار بگیرد. در نتیجه، بخشی از معنای جمله در یک قطعه و ادامه‌ی آن در قطعه‌ای دیگر قرار می‌گیرد. برای کاهش این مشکل، معمولاً از Chunk Overlap استفاده می‌شود؛ یعنی بخشی از انتهای یک Chunk در ابتدای Chunk بعدی نیز تکرار می‌شود. برای مثال، اگر اندازه‌ی هر Chunk برابر با ۵۰۰ کاراکتر و مقدار Overlap برابر با ۵۰ کاراکتر باشد، ۵۰ کاراکتر پایانی هر بخش در بخش بعدی نیز قرار می‌گیرد. به این ترتیب، اطلاعاتی که در مرز میان دو بخش قرار دارند، احتمال بیشتری دارد که ارتباط خود را با بخش بعدی حفظ کنند.



با این حال، در Fixed-Size Chunking، Chunkها همچنان فقط بر اساس اندازه‌ی آن‌ها تعیین می‌شود. بنابراین ممکن است متن درست وسط یک جمله یا پاراگراف قطع شود؛ حتی اگر این نقطه از نظر معنایی، جای مناسبی برای تقسیم متن نباشد.

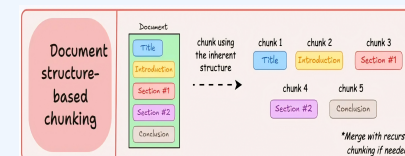
اینجاست که می‌توان به سراغ روشی رفت که بیشتر با ساختار طبیعی متن هماهنگ است: Recursive Chunking. متن با استفاده از چند جداکننده‌ی مختلف و در چند مرحله تقسیم می‌شود. ابتدا این کار با جداکننده‌ای مثل پاراگراف انجام می‌شود. اگر تکه‌های ایجادشده هنوز بیش از حد بزرگ باشند، سیستم سراغ جداکننده‌ی بعدی می‌رود و تقسیم‌بندی را با آن ادامه می‌دهد. برای مثال، یک ترتیب رایج می‌تواند شامل پاراگراف، خط، کلمه و در نهایت کاراکتر باشد. البته این ترتیب ثابت نیست و می‌توان جداکننده‌ها و ترتیب استفاده از آن‌ها را متناسب با ساختار داده تغییر داد.



متن ما همیشه ساده و بدون ساختار نیست. بعضی اسناد ساختار مشخصی دارند و شامل عنوان‌ها، فصل‌ها، زیرعنوان‌ها، فهرست‌ها و جدول‌ها هستند.

در چنین شرایطی، روش‌های Docu-ment-Based Chunking می‌تواند

بن ساختار را هنگام تقسیم سند در نظر بگیرند. برای مثال، در یک مقاله می‌توان عنوان «روش تحقیق» و محتوای مربوط به آن را در یک Chunk قرار داد و بخش «نتایج» را در Chunk دیگری قرار داد. به این ترتیب، ساختار خود سند نیز هنگام تقسیم‌بندی متن حفظ می‌شود.



Title: The Role of Artificial Intelligence in Modern Education	Chunk 1
Introduction	Chunk 2
Artificial intelligence (AI) is reshaping education by providing personalized learning experiences and automating administrative tasks.	
Section 1: Personalized Learning	Chunk 3
AI enables the customization of educational content to meet individual student needs, enhancing engagement and comprehension.	
Section 2: Administrative Automation	Chunk 4
From grading to scheduling, AI tools are streamlining administrative processes, allowing educators to focus more on teaching.	
Conclusion	Chunk 5
The integration of AI in education holds the promise of more efficient learning environments and improved student outcomes.	

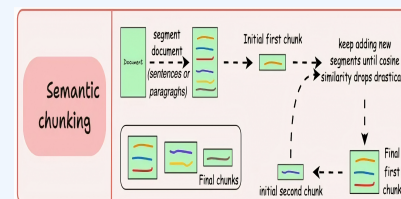
نمونه ای از تقسیم بندی متن بر اساس ساختار و بخش های مختلف سند.

اما همیشه اندازه یا ساختار ظاهری سند برای پیدا کردن بهترین مرزهای تقسیم‌بندی کافی نیست. گاهی چیزی که اهمیت بیشتری دارد، نه شکل قرار گرفتن اطلاعات، بلکه ارتباط

معنایی میان آن‌هاست.

در چنین شرایطی می‌توان به‌جای این‌که مرز Chunkها فقط بر اساس تعداد کلمات یا ساختار ظاهری متن تعیین شود، ارتباط معنایی میان جمله‌ها را نیز در نظر گرفت. یعنی جمله‌هایی که درباره‌ی یک موضوع مشترک صحبت می‌کنند، در کنار یکدیگر قرار بگیرند؛ حتی اگر تعداد کلمات هر بخش با بخش‌های دیگر یکسان نباشد.

به این نوع تقسیم‌بندی، Semantic Chunking گفته می‌شود. در این روش، اندازه‌ی Chunkها می‌تواند متناسب با محتوای هر بخش تغییر کند؛ چون مرز هر بخش را ارتباط معنایی متن تعیین می‌کند، نه یک اندازه‌ی ثابت.



Artificial intelligence is transforming industries by automating processes, enhancing decision-making, and providing insights through data analysis. Machine Learning, a subset of AI, enables systems to learn and improve from experience without explicit programming. Deep Learning, a branch of machine learning, uses neural networks with multiple layers to model complex patterns in data.

نمونه‌ای از تقسیم متن بر اساس ارتباط معنایی بین بخش‌های مختلف.

نشریه سایبرسوتک | ویژه‌نامه

با این حال، این‌که کدام روش نتیجه‌ی بهتری می‌دهد، به نوع و ساختار داده بستگی دارد. روش‌های Chunking نیز فقط به همین موارد محدود نمی‌شوند و روش‌های دیگری هم وجود دارند. حتی در بعضی کاربردها می‌توان چند روش مختلف را با یکدیگر ترکیب کرد.

Embedding: متن چطور به عدد تبدیل می‌شود؟

حالا که متن به بخش‌های کوچک‌تر تقسیم شده، سؤال مهم دیگری پیش می‌آید: سیستم چطور باید بفهمد کدام Chunk از نظر معنایی به سؤال کاربر نزدیک‌تر است؟ برای این کار، هر Chunk به یک بردار عددی یا Embedding تبدیل می‌شود.¹ این بردار، معنای متن را به شکل عددی نمایش می‌دهد.

می‌توانیم Embedding را مثل موقعیت یک متن در یک فضای چندبعدی در نظر بگیریم؛ فضایی که در آن، متن‌هایی که از نظر معنا به یکدیگر نزدیک‌تر هستند، معمولاً در موقعیت‌های نزدیک‌تری نیز قرار می‌گیرند. در واقع، با تبدیل متن به این نمایش عددی، سیستم می‌تواند به‌جای مقایسه‌ی ساده‌ی

1 این تبدیل توسط یک Embedding Model انجام می‌شود؛ مدلی که متن رو دریافت می‌کند و اون رو به یک بردار عددی تبدیل می‌کند. این مدل‌ها با آموزش روی حجم زیادی از داده‌های متنی، الگوها و ارتباط بین واژه‌ها و مفاهیم مختلف رو یاد می‌گیرن. در نتیجه، متن‌هایی با مفهوم مشابه می‌تونن حتی با وجود تفاوت در کلمات، بردارهای نزدیک به یکدیگر داشته باشن.

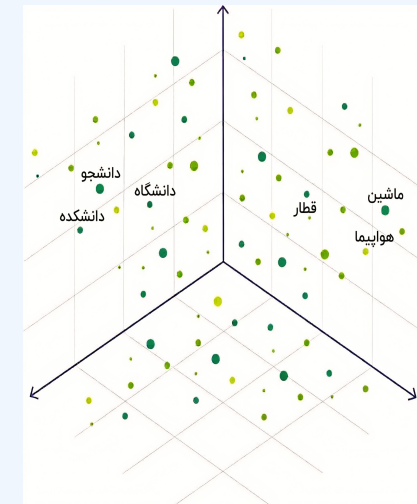
کلمات، میزان شباهت معنایی میان متن‌ها را هم بررسی کند.

Embed-؛ Vector Database

حالا که برای Chunkهای مختلف Embedding ساخته‌ایم، یک سؤال دیگر مطرح می‌شود: این بردارها را کجا نگه داریم و چطور می‌توانیم در میان آن‌ها جست‌وجو کنیم؟ Vector Database دقیقاً برای همین کار به مربوط به هر بخش از سند ذخیره می‌کنند و امکان جست‌وجو بر اساس میزان شباهت میان بردارها را فراهم می‌کنند.

به این ترتیب، وقتی بعداً یک سؤال وارد سیستم می‌شود، می‌توانیم Embedding سؤال را با Embeddingهای ذخیره‌شده مقایسه کنیم و بخش‌هایی از سند را که از نظر معنایی بیشترین شباهت را با سؤال دارند، پیدا کنیم.

در واقع، اگر Embedding را راهی برای تبدیل معنا به عدد در نظر بگیریم، Vector Data- base جایی است که این نمایش‌های عددی را در خود نگه می‌دارد تا هر زمان که لازم باشد، بتوان میان آن‌ها جست‌وجو کرد.



نمایش ساده‌شده embedding در فضای برداری؛ کلمه‌های نزدیک از نظر معنا، در فضای برداری نزدیک تر به هم قرار می‌گیرند.

از میان این همه اطلاعات، کدام بخش به سؤال ما مربوط است؟

پیش از آن‌که به جواب نهایی برسیم، باید یک قدم مهم دیگر برداریم: پیدا کردن اطلاعاتی که واقعاً به سؤال کاربر مربوط است. وقتی حجم داده‌ها زیاد باشد، پیدا کردن این بخش‌های مرتبط دیگر کار ساده‌ای نیست و به روش‌های مناسبی برای جست‌وجو و بازیابی اطلاعات نیاز دارد.

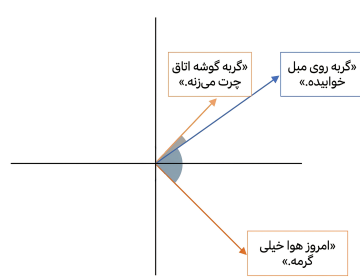
جست‌وجوی معنایی (Seman-tic Search)

یکی از مهم‌ترین روش‌ها برای پیدا کردن اطلاعات مرتبط، جست‌وجوی معنایی یا Semantic Search است.

برخلاف جست‌وجوهای معمولی که بیشتر به دنبال کلمات یکسان یا عبارت‌های مشابه می‌گردند، این روش تلاش می‌کند معنای پشت جمله‌ها را نیز در نظر بگیرد.

فرض کنید کاربری بپرسد: «چطور دوباره وارد حسابم بشم؟» حالا ممکن است در پایگاه داده‌ی ما عنوانی به این صورت وجود داشته باشد: «روش بازیابی رمز عبور حساب کاربری».

در نگاه اول، این دو جمله شباهت چندانی به یکدیگر ندارند؛ کلماتشان متفاوت است و حتی شکل بیانشان هم یکی نیست. اما Semantic Search می‌تواند متوجه شود که هر دو، در اصل درباره‌ی یک موضوع صحبت می‌کنند: بازیابی دسترسی به حساب کاربری. به این ترتیب، جست‌وجوی معنایی به‌جای این‌که فقط به دنبال تطابق کلمات بگردد، تلاش می‌کند ارتباط مفهومی میان سؤال کاربر و اطلاعات موجود را پیدا کند.



شباهت معنایی در فضای برداری؛ هرچه معنای دو جمله به هم نزدیک تر باشد، زاویه بین بردارها کمتر است.

جست‌وجوی ترکیبی (Hybrid Search)

جست‌وجوی معنایی همیشه به‌تنهایی نمی‌تواند بهترین نتیجه را پیدا کند. گاهی وجود یک عبارت دقیق آن‌قدر اهمیت دارد که نمی‌توان از کنار آن گذشت؛ مثلاً وقتی کاربر به دنبال یک کد خطای مشخص مانند Er-404 می‌گردد، لازم است سیستم بتواند دقیقاً همان عبارت را نیز پیدا کند. اینجاست که Hybrid Search به کمک می‌آید.

این روش معمولاً نتایج جست‌وجوی معنایی را با نتایج روش‌های مبتنی بر تطابق کلمات ترکیب می‌کند؛ یعنی هم به معنای سؤال توجه دارد و هم به کلمات و عبارت‌های دقیق موجود در آن. اما با همهی این‌ها، هنوز یک مسئله باقی می‌ماند.

در جست‌وجوی اولیه ممکن است تعداد زیادی نتیجه پیدا شود؛ بعضی از آن‌ها ارتباط

بسیار زیادی با سؤال دارند، بعضی تا حدی مرتبط‌اند و بعضی دیگر اصلاً به پاسخ‌گویی به سؤال کمک نمی‌کنند. حتی گاهی ممکن است یک نتیجه از نظر معنایی شباهت زیادی به سؤال داشته باشد، اما در عمل پاسخ مناسبی برای آن نباشد.

پس سؤال بعدی این است: از میان این همه نتیجه، کدام‌ها باید در اختیار LLM قرار بگیرند؟

بنابراین، باید مشخص کنیم چه تعداد از نتایج جست‌وجو قرار است به مدل داده شوند و کدام نتایج شانس بیشتری برای کمک به تولید پاسخ نهایی دارند.

Top-K؛ چند نتیجه را باید به مدل بدهیم؟

حالا که از میان نتایج جست‌وجو به چندین گزینه رسیده‌ایم، یک سؤال مهم مطرح می‌شود: آیا واقعاً لازم است همهی آن‌ها را در اختیار مدل قرار دهیم؟ Top-K دقیقاً برای پاسخ به همین سؤال است.

در Top-K، از میان تمام نتایج پیداشده، فقط چند مورد از بهترین و مرتبط‌ترین نتایج انتخاب می‌شوند.

مثلاً اگر جست‌وجو صد نتیجه پیدا کرده باشد، طبیعتاً قرار نیست همهی آن‌ها را به مدل بدهیم؛ هم حجم محتوای ورودی زیاد می‌شود و هم ممکن است اطلاعات اضافی

و غیرمرتبط باعث شوند مدل سخت‌تر بتواند روی اطلاعات مهم تمرکز کند.

برای همین می‌توانیم تعیین کنیم که مثلاً Top-5 را نگه داریم؛ یعنی فقط پنج نتیجه‌ی برتر انتخاب شوند. یا Top-3؛ یعنی تنها سه نتیجه‌ی مرتبط‌تر در اختیار مدل قرار بگیرند.

به این ترتیب، تعداد و حجم اطلاعاتی که در ادامه به مدل می‌رسد، کنترل‌شده‌تر می‌شود و مدل به‌جای روبه‌رو شدن با انبوهی از نتایج، با مجموعه‌ای محدودتر از اطلاعات مرتبط سروکار خواهد داشت.

Reranking: آیا بهترین نتایج، به‌ترتیب درست هم قرار گرفته‌اند؟

جست‌وجوی اولیه قرار نیست همیشه نتایج را به بهترین ترتیب مرتب کند؛ هدف اصلی آن این است که در زمان کوتاه، نتایج مرتبط را پیدا کند. اما وقتی قرار است از میان این نتایج، موارد مناسب‌تر را در اختیار مدل قرار دهیم، فقط پیدا کردن اطلاعات مرتبط کافی نیست؛ ترتیب قرار گرفتن آن‌ها هم اهمیت پیدا می‌کند. برای مرتب کردن دقیق‌تر این نتایج، می‌توانیم از Reranking استفاده کنیم.

در این مرحله، یک مدل Reranker نتایج اولیه را با توجه به سؤال کاربر دوباره ارزیابی می‌کند و میزان ارتباط هرکدام را با سؤال، دقیق‌تر مشخص می‌کند. سپس نتایج بر

اساس این ارزیابی دوباره مرتب می‌شوند. به این ترتیب، نتایجی که ارتباط بیشتری با سؤال دارند، در جایگاه بالاتری قرار می‌گیرند و احتمال این‌که اطلاعات مناسب‌تری در اختیار LLM قرار بگیرد، بیشتر می‌شود.



البته Reranking همیشه ضروری نیست و بسته به نوع داده، روش بازبایی و نیاز پروژه می‌تواند مورد استفاده قرار بگیرد. در آخر، بهترین و مرتبط‌ترین اطلاعات به LLM می‌رسند تا مدل بتواند بر اساس آنها پاسخ نهایی را بسازد.

تولید پاسخ؛ مدل با اطلاعات بازبایی‌شده چه می‌کند؟

بعد از بازبایی و مرتب‌سازی اطلاعات، نوبت به تولید پاسخ می‌رسد. در این مرحله، Chunk‌های بازبایی‌شده همراه با سؤال کاربر در اختیار مدل قرار می‌گیرند؛ اما فقط ارائه‌ی این محتوا کافی نیست و باید مشخص کنیم مدل چطور از این اطلاعات استفاده کند.

برای این کار از System Prompt استفاده می‌شود. System Prompt مجموعه‌ای از دستورالعمل‌های کلی است که رفتار مدل را مشخص می‌کند. مثلاً می‌توانیم از مدل بخواهیم پاسخ خودش را بر اساس اطلاعات موجود در Context تولید کند و اگر اطلاعات کافی برای پاسخ‌گویی وجود نداشت، از حدس زدن یا ارائه‌ی اطلاعاتی که در منابع وجود ندارد، خودداری کند.

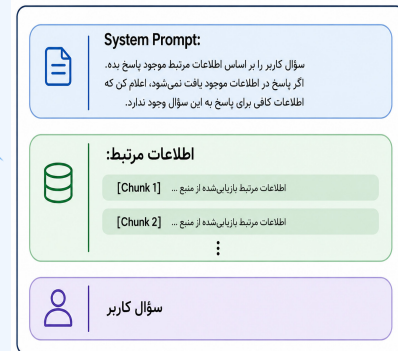
البته این‌که اطلاعات درست را در اختیار مدل قرار دهیم، به این معنی نیست که دیگر احتمال خطا وجود ندارد. مدل هنوز ممکن است در کنار اطلاعات موجود در منابع، اطلاعاتی نادرست یا نامرتب به پاسخ اضافه کند؛ مسئله‌ای که با عنوان توهم (Hallucination) شناخته می‌شود.

بنابراین، این دستورالعمل‌ها می‌توانند احتمال چنین خطاهایی را کاهش دهند، اما نمی‌توانند آن را کاملاً از بین ببرند.

پس در این مرحله، مدل عملاً سه چیز اصلی را در اختیار دارد: دستورالعملی که باید دنبال کند، بخش‌هایی که از منابع بازبایی شده‌اند و سؤال کاربر. مدل با در نظر گرفتن این سه مورد، پاسخ نهایی را تولید می‌کند.

در این ساختار، System Prompt نحوه‌ی رفتار مدل را مشخص می‌کند، Context شامل اطلاعاتی است که سیستم از منابع

خارجی بازبایی کرده و User Query هم پرسش یا درخواست کاربر است. مدل زبانی با استفاده از این اطلاعات، پاسخ نهایی را تولید می‌کند.

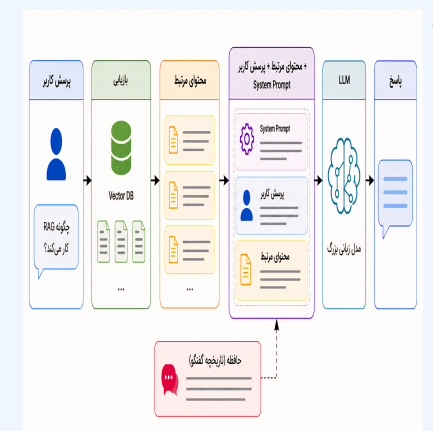


تا اینجا مسیر رسیدن از سؤال کاربر به پاسخ نهایی را دیدیم. اما یک گفت‌وگوی واقعی لزوماً به یک سؤال و یک پاسخ محدود نمی‌شود و کاربر ممکن است سؤال‌های بعدی خودش را بر اساس صحبت‌های قبلی مطرح کند.

برای مثال، فرض کنید یک پژوهشگر چند مقاله‌ی مرتبط با موضوع تحقیق خودش را در اختیار یک چت‌بات قرار داده است. او ابتدا می‌پرسد: «این مقاله چه روشی برای کاهش خطای تشخیص پیشنهاد کرده است؟» کمی بعد سؤال می‌کند: «حالا روش دوم را با مقاله‌ی قبلی مقایسه کن.»



در سؤال دوم، عبارت‌های «روش دوم» و «مقاله قبلی» به گفت‌وگوی قبلی اشاره دارند. بنابراین، سیستم باید بتواند سابقه‌ی گفت‌وگو را هم در اختیار مدل قرار دهد تا منظور کاربر را درست تشخیص دهد. Memory دقیقاً برای نگه‌داشتن چنین سابقه‌ای در طول گفت‌وگو استفاده می‌شود.



پیاده‌سازی RAG؛ حالا چطور آن را بسازیم؟

تا اینجا بخش‌های مختلف RAG را دیدیم؛ از آماده‌سازی و تقسیم اسناد گرفته تا Embedding، بازیابی اطلاعات و در نهایت تولید پاسخ. حالا وقت آن است ببینیم این بخش‌ها در عمل چطور در کنار یکدیگر قرار می‌گیرند و یک سیستم RAG را شکل می‌دهند.

Memory 1 می‌تونه بسته به طراحی سیستم، برای نگهداری اطلاعات در طول گفت‌وگوی جاری (Short-term) یا حفظ اطلاعات برای استفاده در تعاملات بعدی (Long-term) به کار بره.

بدون استفاده از Framework پیاده‌سازی کرد، اما با بیشتر شدن تعداد اجزا، اتصال آن‌ها به یکدیگر می‌تواند پیچیده‌تر شود. اینجااست که Frameworkها می‌توانند کار را ساده‌تر کنند.

این Frameworkها مجموعه‌ای از ابزارها و متدهای آماده برای کار با بخش‌های مختلف RAG در اختیارمان می‌گذارند؛ از آماده‌سازی اسناد و Chunking گرفته تا Embedding و Retrieval و اتصال به مدل زبانی.

در نتیجه، لازم نیست جزئیات هر مرحله را از صفر پیاده‌سازی کنیم و می‌توانیم اجزای مختلف را راحت‌تر به یکدیگر متصل کنیم. البته استفاده از چنین Frameworkهایی یک نکته‌ی مهم هم دارد. بخشی از جزئیات و نحوه‌ی پیاده‌سازی، پشت ابزارهای آماده پنهان می‌شود. در نتیجه، کار با این ابزارها ساده‌تر خواهد شد، اما کنترل و دسترسی مستقیم به بعضی از این جزئیات نیز کمتر می‌شود.

انتخاب Framework

همان‌طور که گفتیم، برای پیاده‌سازی یک سیستم RAG، استفاده از یک Framework الزامی نیست و می‌توان هر یک از مراحل آن را به‌صورت مستقیم پیاده‌سازی کرد. در میان Frameworkهای موجود، LangChain و

نشریه سایبرسوتک | ویژه‌نامه

LlamaIndex از شناخته‌شده‌ترین گزینه‌ها برای ساخت سیستم‌های مبتنی بر RAG هستند. هر دو Framework بسیاری از مراحل یک Pipeline کامل RAG را پوشش می‌دهند، اما رویکرد متفاوتی نسبت به مسئله دارند.

LangChain؛ اتصال اجزای مختلف یک سیستم مبتنی بر مدل زبانی

این فریم‌ورک بیشتر روی ساخت و مدیریت برنامه‌هایی تمرکز دارد که مدل زبانی یکی از اجزای اصلی آن‌هاست. منظور از این برنامه‌ها، سیستم‌هایی است که در آن‌ها مدل زبانی تنها وظیفه‌ی تولید پاسخ را بر عهده ندارد، بلکه باید با بخش‌های مختلف دیگر نیز تعامل داشته باشد.

یک دستیار هوشمند را در نظر بگیرید که برای پاسخ‌گویی به درخواست‌های کاربران، ممکن است نیاز داشته باشد چند مرحله‌ی مختلف را طی کند. ابتدا درخواست را تحلیل کرده و مشخص می‌کند برای رسیدن به پاسخ به چه اطلاعاتی نیاز دارد. سپس بسته به نوع درخواست، اطلاعات موردنیاز را از منابع مختلفی مانند اسناد موجود یا پایگاه داده به دست می‌آورد. در نهایت، اطلاعات جمع‌آوری‌شده در اختیار مدل زبانی قرار می‌گیرد تا پاسخ نهایی تولید شود.

نشریه سایبرسوتک | ویژه‌نامه

مجموعه‌ی این مراحل که برای رسیدن به یک پاسخ انجام می‌شوند، یک Workflow را تشکیل می‌دهد.

در چنین سیستم‌هایی، مسئله فقط پیدا کردن اطلاعات مرتبط نیست؛ بلکه هماهنگی بین بخش‌های مختلف نیز اهمیت پیدا می‌کند. Langchain با فراهم کردن ابزارهایی برای تعریف و مدیریت این ارتباط‌ها، ساخت چنین Workflowهایی را ساده‌تر می‌کند.

البته این به معنی محدود بودن آن به سیستم‌های پیچیده نیست و می‌توان از آن برای پیاده‌سازی RAGهای ساده‌ی مبتنی بر اسناد نیز استفاده کرد؛ همانند نمونه‌ای که در ادامه این بخش پیاده‌سازی می‌کنیم.

LlamaIndex؛ تمرکز بیشتر روی داده و بازیابی اطلاعات

در مقابل، LlamaIndex بیشتر روی سازمان‌دهی اطلاعات و ساخت سیستم‌های مبتنی بر داده تمرکز دارد. ایده‌ی اصلی این Framework این است که چگونه حجم زیادی از اطلاعات را به شکلی مناسب برای استفاده توسط مدل‌های زبانی آماده کنیم.

برای مثال، فرض کنید یک شرکت هزاران سند شامل مستندات فنی، گزارش‌ها، قراردادهای و



فایل‌های PDF دارد و می‌خواهد سیستمی بسازد که بتواند به سؤال‌های کاربران درباره‌ی این اطلاعات پاسخ دهد.

در چنین پروژه‌ای، فقط اتصال یک مدل زبانی به یک Vector Database کافی نیست؛ بلکه نحوه‌ی سازمان‌دهی اسناد، حفظ ارتباط بین بخش‌های مختلف و روش بازیابی اطلاعات اهمیت زیادی دارد.

LlamaIndex با تمرکز روی مفاهیمی مانند Document، Node و Index، ابزارهایی برای ساختاردهی بهتر اطلاعات و بازیابی دقیق‌تر بخش‌های مرتبط فراهم می‌کند.

Document همان منبع اصلی اطلاعات است؛ برای مثال یک فایل PDF، صفحه‌ی وب یا یک گزارش.

در ادامه همان Chunkهایی که در مرحله‌ی آماده‌سازی برای Embedding و جست‌وجو ایجاد می‌شوند، در این فریم‌ورک معمولاً در قالب Node مدیریت می‌شوند. Node را می‌توان ساختاری مشابه یک Chunk در نظر گرفت که علاوه بر متن اصلی، اطلاعات جانبی و ارتباطات مربوط به آن را نیز نگهداری می‌کند.

پس از ایجاد Nodeها، نوبت به سازمان‌دهی آن‌ها در قالب یک Index است.

Index ساختاری قابل جست‌وجو روی اطلاعات ایجاد می‌کند که به سیستم کمک

می‌کند هنگام دریافت سؤال، به جای بررسی تمام داده‌ها، سریع‌تر بخش‌های مرتبط را پیدا کند.

بنابراین، تفاوت اصلی این دو Framework در نوع مسئله‌ای است که روی آن تمرکز می‌کنند.

LangChain برای مدیریت workflow و اتصال اجزای مختلف یک سیستم مبتنی بر مدل زبانی مناسب‌تر است، در حالی که LlamaIndex بر آماده‌سازی و سازمان‌دهی اطلاعات برای استفاده‌ی بهتر مدل زبانی تمرکز دارد.

در نهایت، هیچ‌کدام از این دو Framework به‌صورت مطلق بهتر از دیگری نیستند و انتخاب بین آن‌ها به نوع پروژه بستگی دارد. اگر RAG تنها یکی از بخش‌های یک سیستم بزرگ‌تر باشد، مدیریت Workflow و اتصال اجزا اهمیت بیشتری پیدا می‌کند. اما اگر هدف اصلی، ساخت یک سیستم جست‌وجو و پرسش‌وپاسخ روی مجموعه‌ای از اسناد باشد، سازمان‌دهی اطلاعات و کیفیت Retrieval اهمیت بیشتری خواهد داشت.

همچنین باید توجه داشت که در پروژه‌های ساده، ممکن است استفاده از هیچ فریم‌ورکی ضروری نباشد و ترکیب مستقیم یک مدل

Embedding، یک Vector Database و یک مدل زبانی، نیاز سیستم را برطرف کند.



نگاهی به پیاده‌سازی

حالا که با ایده‌ی کلی RAG و نقش Frame-workها آشنا شدیم، وقت آن است سراغ یک نمونه‌ی ساده از پیاده‌سازی RAG با LangChain برویم و ببینیم هر بخش از کد دقیقاً چه کاری انجام می‌دهد. البته LlamaIndex نیز می‌تواند برای چنین سناریویی مورد استفاده قرار گیرد، اما در این مثال، هدف صرفاً نمایش مراحل اصلی پیاده‌سازی RAG و نحوه‌ی اتصال این بخش‌ها به یکدیگر است. برای ساخت این نمونه، چند ابزار و کتابخانه به کار می‌بریم که هرکدام وظیفه‌ی مشخصی دارند:

ابزار	کاربرد
PyPDFium2Loader	خواندن محتوای فایل PDF
RecursiveCharacterTextSplitter	تقسیم سند به Chunkهای کوچک‌تر
HuggingFaceEmbeddings	تبدیل متن Chunkها به بردارهای عددی
FAISS	ذخیره و جست‌وجوی بردارها
ChatOpenAI	ارتباط با مدل زبانی و تولید پاسخ

نشریه سایبرسوتک | ویژه‌نامه

```
1 pip install langchain \
2 langchain-community \
3 langchain-text-splitters \
4 langchain-huggingface \
5 langchain-openai \
6 faiss-cpu
7
```

نحوه نصب ابزارهای مورد نیاز

همچنین برای تولید پاسخ، به یک مدل زبانی نیاز داریم. در این نمونه از ChatOpenAI استفاده می‌کنیم که امکان اتصال به سرویس‌های سازگار با API های OpenAI را فراهم می‌کند. برای برقراری این ارتباط، به یک API key نیاز داریم که درخواست‌های برنامه به سرویس مدل زبانی را مجاز می‌کند. می‌توان این API Key را از سرویس‌هایی مانند OpenRouter دریافت کرد.

آماده‌سازی سند

اولین قدم این است که محتوای فایل PDF را وارد Pipeline کنیم. برای این کار از PyPDF2 و Fium2Loader استفاده می‌کنیم. این ابزار فایل PDF را می‌خواند و محتوای آن را به ساختار Document در LangChain تبدیل می‌کند.

تقسیم سند

همان‌طور که قبل‌تر گفتیم باید سند را به Chunk‌های کوچک‌تر تقسیم کنیم. برای تقسیم سند در این پیاده‌سازی از `Recur-siveCharacterTextSplitter` استفاده شده است. مقدار `chunk_size` برابر با ۱۰۰۰ و `chunk_overlap` برابر با ۱۵۰ در نظر گرفته شده است

اما این مدت دقیقاً چطور متن را تقسیم می‌کند؟ جداکننده‌های پیش فرض `RecursiveCharacterTextSplitter` به ترتیب `["\\n\\n", "\\n", " ", ""]` هستند؛ یعنی مدت ابتدا تلاش می‌کند متن را از مرز پاراگراف‌ها جدا کند. اگر بخش ایجادشده هنوز بیش از اندازه بزرگ باشد، سراغ جداکننده‌ی بعدی، یعنی خط، می‌رود. اگر باز هم اندازه‌ی بخش مناسب نباشد، تقسیم‌بندی با فاصله و در نهایت با کاراکتر ادامه پیدا می‌کند.

البته این جداکننده‌ها ثابت نیستند و اگر ساختار سند به شکل دیگری باشد، می‌توان با استفاده از پارامتر `separators` آن‌ها را تغییر داد و نحوه‌ی تقسیم متن را متناسب با ساختار سند تنظیم کرد.

Embedding و ذخیره‌سازی؛ بردارها کجا قرار می‌گیرند؟

برای تبدیل Chunkها به بردارهای عددی از یک مدل چندزبانه استفاده شده که از متن فارسی و انگلیسی پشتیبانی می‌کند.

مدل‌های Embedding مختلفی وجود دارند و انتخاب بین آن‌ها می‌تواند به زبان، نوع داده و نیاز پروژه بستگی داشته باشد. برای ذخیره‌سازی بردارها در این پیاده‌سازی از FAISS استفاده شده است. FAISS بیشتر روی جست‌وجوی سریع بین بردارهای مشابه تمرکز دارد، اما گزینه‌های دیگری هم وجود دارند. برای مثال، Chroma امکانات بیشتری برای مدیریت بردارها،¹ Metadata و فیلتر کردن نتایج فراهم می‌کند. بنابراین، بسته به نیاز پروژه، می‌توان به جای FAISS از ابزارهایی مثل Chroma نیز استفاده کرد.

Retrieval؛ کدام بخش‌های سند به سؤال مربوط‌اند؟

بعد از دریافت سؤال کاربر، سیستم باید بخش‌های مرتبط با آن را از میان سند پیدا کند.



```
1 related_chunks = db.similarity_search(query, k=5)
```

1 Metadata : اطلاعات اضافی مرتبط با هر Chunk، مثل نام سند، شماره صفحه، فصل یا بخش، که می‌تونه برای مدیریت یا فیلتر کردن نتایج بازایی مورد استفاده قرار بگیره.

متد `similarity_search` نزدیک ترین `Chunk` ها به سؤال کاربر را بر اساس میزان شباهت میان بردارها بازیابی می‌کند. مقدار `k` تعداد نتایج موردنظر را مشخص می‌کند؛ در اینجا، پنچ `Chunk` برتر انتخاب می‌شوند.

ساخت Context و تولید پاسخ؛ اطلاعات چطور به مدل می‌رسند؟

پس از بازیابی، Chunkهای بازیابی شده در قالب یک Context در کنار سؤال کاربر قرار می‌گیرند و به مدل زبانی داده می‌شوند.

[illegible]

prompt_template متن درخواست مدل را تشکیل می‌دهد و اطلاعات منبع، سؤال کاربر و قوانین پاسخ‌گویی را در کنار هم قرار می‌دهد. داخل پرامپت از مدل خواسته شده است پاسخ خود را بر اساس اطلاعات منبع بنویسد و اگر اطلاعات کافی برای پاسخ وجود نداشت، از حدس زدن خودداری کند. این دستورات عمل‌ها همان مفاهیمی هستند که در بخش‌های قبل درباره‌ی نحوه‌ی استفاده‌ی مدل از Context به آن‌ها اشاره کردیم.

```

1 print("Embeddings در حال بارگذاری است...")
2 embeddings = HuggingFaceEmbedder(
3     model_name="sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2"
4 )
5
6 print("..... در حال ساخت پایگاه داده پردازش...")
7 db = FAISS.from_documents(
8     chunks,
9     embedding=embeddings
10 )

```

```
1 text_splitter = RecursiveCharacterTextSplitter(  
2     chunk_size=1000,  
3     chunk_overlap=150  
4 )  
5 chunks = text_splitter.split_documents(documents)
```

1. An Introduction to Formal Languages and Automata, 5th Edition – Peter Linz

نشریه سایبرسو تک | ویژه‌نامه

نشریه سایبرسوتک | ویژه‌نامه



دانشگاه صنعتی شیراز

انجمن علمی کامپیوتر
دانشگاه صنعتی شیراز



</>

</>

پایان

